

lua-xlsx - English documentation

Julien Freyermuth

Contents

1	lua-xlsx - English documentation	2
1.1	Contents	2
1.2	Architecture and Babet integration	2
1.3	Conventions	2
1.4	Writing XLSX files	3
1.4.1	xlsx.new([opts]) -> workbook	3
1.4.2	workbook:add_sheet([name]) -> sheet	3
1.4.3	sheet:write(row, col, value [, style]) -> sheet	3
1.4.4	sheet:append_row(values [, style])	3
1.4.5	sheet:write_rows(matrix [, style])	3
1.5	Styles, structure, layout, hyperlinks and formulas	4
1.5.1	xlsx.VERSION	4
1.5.2	xlsx.style([opts]) -> style	4
1.5.3	xlsx.is_style(value) and xlsx.style_options(style)	5
1.5.4	sheet:set_style(row, col, style) -> sheet	5
1.5.5	Column widths and row heights	5
1.5.6	Frozen panes	5
1.5.7	Automatic filters	6
1.5.8	Merged cells	6
1.5.9	Hyperlinks	6
1.5.10	Formulas	6
1.5.11	Complete 1.2.0 example	6
1.5.12	workbook:save(path [, opts])	7
1.5.13	xlsx.write_rows(path, matrix [, opts])	8
1.6	1900 and 1904 dates	8
1.7	Reading XLSX files	8
1.7.1	xlsx.open(path [, opts])	8
1.7.2	Workbook methods	9
1.7.3	Values and formulas	9
1.7.4	Styles	9
1.7.5	Layout	9
1.7.6	Merged cells	9
1.7.7	Hyperlinks	9
1.7.8	Dimensions and row iteration	9
1.8	ZIP validation and limits	10
1.9	DataFrame	10
1.9.1	Construction	10
1.9.2	Introspection	11
1.9.3	Non-destructive transformations	11
1.9.4	Grouping and aggregation	11
1.9.5	Output	12
1.10	Error contract	12
1.11	Tests and interoperability	12
1.12	Building the PDFs	12
1.13	Known limitations	13

1 lua-xlsx - English documentation

Reference for the `xlsx` and `dataframe` modules. Babet with Lua 5.5 is the primary target, while the pure-Lua core remains compatible with standard Lua 5.3+.

Documentation for lua-xlsx 1.2.0.

1.1 Contents

- [Architecture and Babet integration](#)
 - [Conventions](#)
 - [Writing XLSX files](#)
 - [Styles, structure, layout, hyperlinks and formulas](#)
 - [1900 and 1904 dates](#)
 - [Reading XLSX files](#)
 - [ZIP validation and limits](#)
 - [DataFrame](#)
 - [Error contract](#)
 - [Tests and interoperability](#)
 - [Building the PDFs](#)
 - [Known limitations](#)
-

1.2 Architecture and Babet integration

`xlsx.lua` and `dataframe.lua` require no external Lua module. When the global `babet` table is available, `xlsx.lua` automatically uses:

Babet function	lua-xlsx use
<code>babet.writeFileAtomic</code>	atomic and durable XLSX publication
<code>babet.archive.test</code>	full ZIP validation before parsing
<code>babet.fileSize</code>	file-size check before loading into memory
<code>babet.crc32</code>	native CRC-32 calculation

If one function is absent, the pure-Lua path is retained for that operation.

To disable Babet for one call:

```
assert(wb:save("report.xlsx", { use_babet = false })))
```

```
local read, err = xlsx.open("report.xlsx", {  
    use_babet = false,  
})  
assert(read, err)
```

This is mostly useful for testing the fallback. Production code running in Babet should normally keep the default true.

1.3 Conventions

- `sheet:write(row, col, value [, style])` and `sheet:read(row, col)` use **0-based** indices. Cell A1 is (0, 0).
- `sheet:rows()` yields **1-based** Lua arrays. `row[1]` is column A and `row.n` is the width.
- An empty cell is `nil`. Boolean `false` never collapses to `nil`.
- Dates are returned as ISO 8601 strings.
- XLSX bounds are enforced: rows 0..1048575, columns 0..16383.
- Written strings must be valid UTF-8 and contain only XML 1.0 characters.
- `DataFrame` transformations return newly copied row records.

1.4 Writing XLSX files

1.4.1 `xlsx.new([opts]) -> workbook`

Option:

Option	Default	Values
<code>date_system</code>	"1900"	"1900" or "1904"

```
local wb1900 = xlsx.new()
local wb1904 = xlsx.new({ date_system = "1904" })
```

Unknown options are rejected.

1.4.2 `workbook:add_sheet([name]) -> sheet`

The default name is SheetN.

Rules:

- 1 to 31 **Unicode characters**, not bytes;
- valid UTF-8 and XML 1.0;
- none of `:`, `\`, `/`, `?`, `*`, `[` or `]`;
- no leading or trailing apostrophe;
- no exact duplicate or ASCII case-only duplicate.

```
local wb = xlsx.new()
local data = wb:add_sheet("Data")
local unicode = wb:add_sheet(string.rep("é", 20))
```

1.4.3 `sheet:write(row, col, value [, style]) -> sheet`

Accepted values: string, finite number, boolean, date value, formula value or `nil`. The optional style must be created by `xlsx.style`.

```
local sh = xlsx.new():add_sheet("Example")
sh:write(0, 0, "Text")
sh:write(0, 1, 42)
sh:write(0, 2, 3.14)
sh:write(0, 3, false)
sh:write(0, 4, xlsx.date(2026, 7, 18))
sh:write(0, 5, xlsx.formula("SUM(B1:C1)"))
```

NaN, infinities, unsupported types and out-of-range indices raise a Lua error.

1.4.4 `sheet:append_row(values [, style])`

```
sh:append_row({ "Alice", 30, true })
sh:append_row({ "Bob", 25, false })
```

1.4.5 `sheet:write_rows(matrix [, style])`

```
sh:write_rows({
  { "Name", "Score" },
  { "Alice", 18 },
  { "Bob", 15 },
})
```

1.5 Styles, structure, layout, hyperlinks and formulas

Version 1.2.0 writes these elements and also exposes the supported subset through the reading API. style, formula, and hyperlink objects are immutable.

1.5.1 `xlsx.VERSION`

```
assert(xlsx.VERSION == "1.2.0")
```

1.5.2 `xlsx.style([opts]) -> style`

A style must be created with `xlsx.style`; raw option tables are rejected by `write()` and `set_style()`.

Option	Type	Values
<code>bold</code>	boolean	bold font
<code>italic</code>	boolean	italic font
<code>underline</code>	string	none, single, double
<code>strike</code>	boolean	strikethrough
<code>font_name</code>	string	non-empty UTF-8 name, up to 255 bytes
<code>font_size</code>	number	1 to 409 points
<code>font_color</code>	string	RGB RRGGBB, #RRGGBB, or ARGB AARRGGBB
<code>fill_color</code>	string	solid RGB or ARGB fill
<code>horizontal</code>	string	left, center, right, justify
<code>vertical</code>	string	top, center, bottom, justify
<code>wrap_text</code>	boolean	wrap text
<code>number_format</code>	string	alias or Excel format code
<code>border</code>	table	left, right, top, bottom sides

Six-digit RGB colors automatically receive an opaque FF alpha channel.

Bold and italic:

```
local emphasis = xlsx.style({ bold = true, italic = true })
sh:write(0, 0, "Important", emphasis)
```

Font name and size:

```
local title = xlsx.style({
  font_name = "Liberation Sans",
  font_size = 16,
})
sh:write(1, 0, "Title", title)
```

Underlining and strikethrough:

```
sh:write(2, 0, "Single", xlsx.style({ underline = "single" }))
sh:write(3, 0, "Double", xlsx.style({ underline = "double" }))
sh:write(4, 0, "Old value", xlsx.style({ strike = true }))
```

Font and fill colors:

```
local colored = xlsx.style({
  font_color = "FFFFFF",
  fill_color = "4472C4",
})
```

Alignment and wrapping:

```
local wrapped = xlsx.style({
  horizontal = "center",
  vertical = "center",
  wrap_text = true,
})
```

Number-format aliases:

Alias	Generated Excel code
general	no custom format
integer	0
decimal	0.00
percent	0.00%
currency_eur	#,##0.00 "€"
currency_usd	\$#,##0.00
date	yyyy-mm-dd
datetime	yyyy-mm-dd hh:mm:ss

```
sh.write(0, 1, 42, xlsx.style({ number_format = "integer" }))
sh.write(1, 1, 3.14159, xlsx.style({ number_format = "decimal" }))
sh.write(2, 1, 0.125, xlsx.style({ number_format = "percent" }))
sh.write(3, 1, 19.90, xlsx.style({ number_format = "currency_eur" }))
```

Custom code:

```
sh.write(4, 1, 12.3456, xlsx.style({ number_format = '0.000 "kg"' }))
```

Date values automatically receive a date format if the style does not specify one.

1.5.2.1 Borders Each side is a table with style and optional color. Supported styles are thin, medium, thick, dashed, dotted, and double.

```
local framed = xlsx.style({
  border = {
    left = { style = "thin", color = "808080" },
    right = { style = "thin", color = "808080" },
    top = { style = "double", color = "000000" },
    bottom = { style = "double", color = "000000" },
  },
})
```

1.5.3 xlsx.is_style(value) and xlsx.style_options(style)

```
assert(xlsx.is_style(framed))
local opts = xlsx.style_options(framed)
```

style_options() returns a deep copy.

1.5.4 sheet:set_style(row, col, style) -> sheet

```
sh:set_style(0, 0, xlsx.style({ bold = true }))
sh:set_style(1, 0, xlsx.style({ fill_color = "E2F0D9" }))
sh:set_style(0, 0, nil)
```

1.5.5 Column widths and row heights

```
sh:set_column_width(0, 24) -- 0.1 to 255
sh:set_row_height(0, 28) -- 0.1 to 409.5 points
```

1.5.6 Frozen panes

```
sh:freeze_panes(1, 0)
sh:freeze_panes(0, 1)
sh:freeze_panes(1, 1)
sh:freeze_panes(0, 0)
```

1.5.7 Automatic filters

```
sh:set_auto_filter()  
sh:set_auto_filter("A1:D100")  
sh:set_auto_filter(false)
```

1.5.8 Merged cells

```
sh:write(0, 0, "Report")  
sh:merge_cells("A1:D1")  
sh:merge_cells(2, 0, 2, 3) -- A3:D3, zero-based coordinates  
sh:unmerge_cells("A1:D1")
```

Single-cell, reversed, and overlapping merges are rejected. Only the top-left cell keeps its value, style, and hyperlink.

1.5.9 Hyperlinks

External hyperlink value:

```
sh:write(0, 0, xlsx.hyperlink(  
    "https://example.com/?a=1&b=2",  
    "Open website",  
    { tooltip = "External website" }  
))
```

Internal hyperlink:

```
sh:write(1, 0, xlsx.hyperlink(  
    "'Details'!A1",  
    "View details",  
    { internal = true }  
))
```

Apply or remove a hyperlink on an existing cell:

```
sh:write(2, 0, "Documentation")  
sh:set_hyperlink(2, 0, "https://example.com/docs")  
sh:set_hyperlink(3, 0, "'Summary'!B2", { internal = true })  
sh:remove_hyperlink(2, 0)
```

1.5.10 Formulas

```
sh:write(3, 1, xlsx.formula("SUM(B1:B3)"))  
sh:write(3, 2, xlsx.formula("=AVERAGE(C1:C3)"))  
sh:write(4, 1, xlsx.formula("SUM(B2:B4)", 42.5))
```

The optional cached value may be a string, number, or boolean. It is not verified against the formula. lua-xlsx does not calculate formulas.

1.5.11 Complete 1.2.0 example

```
local xlsx = require("xlsx")  
local wb = xlsx.new()  
local sh = wb:add_sheet("Sales")  
  
local title = xlsx.style({  
    bold = true,  
    underline = "double",  
    font_name = "Liberation Sans",  
    font_size = 16,  
    font_color = "FFFFFF",  
    fill_color = "4472C4",  
    horizontal = "center",
```

```

    border = { bottom = { style = "double", color = "1F4E78" } },
})
local header = xlsx.style({
    bold = true,
    fill_color = "D9EAF7",
    horizontal = "center",
    border = {
        left = { style = "thin", color = "808080" },
        right = { style = "thin", color = "808080" },
        top = { style = "thin", color = "808080" },
        bottom = { style = "thin", color = "808080" },
    },
})
local money = xlsx.style({ number_format = "currency_eur" })

sh:write(0, 0, "Sales report", title)
sh:merge_cells("A1:E1")
sh:append_row({ "Product", "Price", "Qty", "Total", "Link" }, header)
sh:append_row({ "Keyboard", 39.90, 2 })
sh:append_row({ "Mouse", 24.50, 3 })
sh:write(2, 1, 39.90, money)
sh:write(3, 1, 24.50, money)
sh:write(2, 3, xlsx.formula("B3*C3", 79.80), money)
sh:write(3, 3, xlsx.formula("B4*C4", 73.50), money)
sh:write(2, 4, xlsx.hyperlink("https://example.com/keyboard", "Product"))
sh:set_column_width(0, 24)
sh:set_row_height(0, 30)
sh:freeze_panes(2, 1)
sh:set_auto_filter("A2:E4")
assert(wb:save("sales.xlsx"))

```

1.5.12 workbook:save(path [, opts])

Option	Default	Effect
overwrite	true	replace an existing file
durable	true	request durable synchronization in Babet
permissions	0644	final Babet file mode
use_babet	true	use writeFileAtomic when available

Simple write:

```

local ok, err = wb:save("report.xlsx")
assert(ok, err)

```

Refuse overwrite:

```

local ok, err = wb:save("report.xlsx", {
    overwrite = false,
})

```

Rebuildable cache:

```

assert(wb:save("cache.xlsx", {
    durable = false,
}))

```

Private permissions:

```

assert(wb:save("private.xlsx", {
    permissions = tonumber("600", 8),
}))

```

Combined example:

```
local ok, err = wb:save("export/report.xlsx", {
    overwrite = true,
    durable = true,
    permissions = tonumber("640", 8),
    use_babet = true,
})
assert(ok, err)
```

With Babet, this calls `babet.writeFileAtomic`. Standard Lua writes a temporary file in the same directory, checks write, flush and close, then renames it. The fallback is atomic on ordinary POSIX filesystems but does not duplicate all of Babet's descriptor confinement and `fsync` guarantees.

1.5.13 `xlsx.write_rows(path, matrix [, opts])`

Options are `sheet`, `date_system`, `overwrite`, `durable`, `permissions` and `use_babet`.

```
assert(xlsx.write_rows("result.xlsx", {
    { "x", "y" },
    { 1, 2 },
    { 3, 4 },
}, {
    sheet = "Results",
    date_system = "1900",
    overwrite = true,
}))
```

1.6 1900 and 1904 dates

```
sh:write(0, 0, xlsx.date(2026, 7, 18))
sh:write(0, 1, xlsx.datetime(2026, 7, 18, 13, 45, 30))
```

The constructors validate real calendar dates, leap years, years 1..9999, hours 0..23, minutes and seconds 0..59, and integer argument types.

Helpers:

```
local serial1900 = xlsx.date_to_serial(2026, 7, 18)
local serial1904 = xlsx.date_to_serial(2026, 7, 18, 0, 0, 0, true)

print(xlsx.serial_to_iso(serial1900, false))
print(xlsx.serial_to_iso(serial1904, false, true))
```

The writer emits the correct serial for the workbook's `date_system`. The reader automatically detects `<workbookPr date1904="1"/>`.

1.7 Reading XLSX files

1.7.1 `xlsx.open(path [, opts])`

```
local wb, err = xlsx.open("report.xlsx")
assert(wb, err)
```

Opening checks file size, optionally validates the archive with Babet, verifies ZIP metadata and CRCs, and parses the required XML parts.

1.7.2 Workbook methods

```
print(wb:date_system())
for _, name in ipairs(wb:sheet_names()) do print(name) end
local sheet = assert(wb:sheet("Data"))
```

sheet() also accepts a one-based sheet index.

1.7.3 Values and formulas

sheet:read(row, col) keeps the historical contract and returns the cached cell value. A formula without a cache therefore returns nil.

```
local value = sheet:read(0, 0)
local formula = sheet:get_formula(3, 1)
if formula then
    print(formula.expression, formula.cached_value, formula.formula_type)
end
```

Formula objects also expose ref and shared_index when present. A secondary shared formula may have no resolved expression; it can be inspected but cannot be written directly.

1.7.4 Styles

```
local style = sheet:get_style(0, 0)
if style then
    print(style.bold, style.font_name, style.font_size)
    print(style.number_format)
    print(style.border and style.border.bottom and style.border.bottom.style)
end
```

The returned style may be passed to write() or set_style(). Only the supported subset is exposed; themes, indexed colors, diagonal borders, and advanced variants may be omitted.

1.7.5 Layout

```
print(sheet:get_column_width(0))
print(sheet:get_row_height(0))
local rows, cols = sheet:get_frozen_panes()
print(rows, cols)
print(sheet:get_auto_filter())
```

1.7.6 Merged cells

```
for _, range in ipairs(sheet:merged_cells()) do print(range) end
```

1.7.7 Hyperlinks

```
local link = sheet:get_hyperlink(2, 4)
if link then print(link.target, link.internal, link.tooltip, link.ref) end
for _, item in ipairs(sheet:hyperlinks()) do print(item.ref, item.target) end
```

1.7.8 Dimensions and row iteration

```
local maxrow, maxcol = sheet:dims()
for row in sheet:rows() do
    for col = 1, row.n do print(row[col]) end
end
```

rows() iterates over cached values and preserves intermediate empty rows.

1.8 ZIP validation and limits

Option	Default	Maximum
max_file_size	256 MiB	no extra internal ceiling
max_entries	10,000	100,000
max_entry_size	64 MiB	8 GiB
max_total_size	512 MiB	64 GiB
max_path_length	4,096 bytes	1 MiB
max_total_name_bytes	16 MiB	64 MiB
max_compression_ratio	200	1,000,000,000
use_babet	true	boolean
validate_archive	true	boolean

One example per limit:

```
xlsx.open("a.xlsx", { max_file_size = 32 * 1024 * 1024 })
xlsx.open("a.xlsx", { max_entries = 2000 })
xlsx.open("a.xlsx", { max_entry_size = 16 * 1024 * 1024 })
xlsx.open("a.xlsx", { max_total_size = 128 * 1024 * 1024 })
xlsx.open("a.xlsx", { max_path_length = 1024 })
xlsx.open("a.xlsx", { max_total_name_bytes = 1024 * 1024 })
xlsx.open("a.xlsx", { max_compression_ratio = 100 })
```

Combined untrusted-file profile:

```
local wb, err = xlsx.open("import.xlsx", {
    max_file_size = 64 * 1024 * 1024,
    max_entries = 5000,
    max_entry_size = 16 * 1024 * 1024,
    max_total_size = 128 * 1024 * 1024,
    max_path_length = 2048,
    max_total_name_bytes = 2 * 1024 * 1024,
    max_compression_ratio = 100,
    use_babet = true,
    validate_archive = true,
})
assert(wb, err)
```

The Lua reader rejects multi-disk ZIPs, ZIP64, encryption, unsupported methods, unsafe or duplicate names, inconsistent local headers or data descriptors, overlapping entries, size mismatches, CRC failures and truncated DEFLATE data.

1.9 DataFrame

A DataFrame contains unique, non-empty string columns and rows records.

1.9.1 Construction

1.9.1.1 DF.from_rows(matrix [, opts]) opts.header uses the first row as names. opts.columns supplies explicit names when header is false.

```
local d = DF.from_rows({
    { "name", "age" },
    { "Alice", 30 },
    { "Bob", 25 },
}, { header = true })
```

Duplicate column names are rejected.

1.9.1.2 DF.from_records(records [, columns])

```
local d = DF.from_records({
  { name = "Alice", age = 30 },
  { name = "Bob", age = 25 },
}, { "name", "age" })
```

Without columns, string keys are inferred and sorted.

1.9.1.3 DF.from_sheet(sheet [, opts])

```
local d = DF.from_sheet(assert(wb:sheet("Data")), {
  header = true,
})
```

1.9.2 Introspection

```
print(d:nrow(), d:ncol())
print(table.concat(d:colnames(), ", "))
local ages = d:column("age")
```

d:iter() yields copies:

```
for row in d:iter() do
  row.age = 0 -- does not modify d
end
```

1.9.3 Non-destructive transformations

```
local adults = d:filter(function(row) return row.age >= 18 end)
local names = d:select("name")
local renamed = d:rename({ age = "years" })
local doubled = d:mutate("double_age", function(row) return row.age * 2 end)
local asc = d:sort("age")
local desc = d:sort("age", { desc = true })
local first = d:head(10)
local last = d:tail(10)
```

filter and mutate callbacks receive a copy. filter, sort, head, tail and iter do not share row tables with the source. Rename collisions and repeated selections are rejected. Sort is stable and keeps nil last.

1.9.4 Grouping and aggregation

```
local grouped = sales:groupby("city", "product"):agg({
  total = { "sum", "amount" },
  average = { "mean", "amount" },
  minimum = { "min", "amount" },
  maximum = { "max", "amount" },
  first = { "first", "amount" },
  last = { "last", "amount" },
  count = { "count" },
})
```

Functions: sum, mean/avg, min, max, count, first, last.

Group keys support nil, booleans, binary strings and numbers. The internal key encoding is type- and length-prefixed, so embedded separators or NUL bytes cannot merge distinct groups.

```
local counts = d:groupby("city"):count()
```

1.9.5 Output

```
local records = d:to_records()
local rows = d:to_rows()
local no_header = d:to_rows({ header = false })
print(d:tostring(20))
d:show(20)
```

1.10 Error contract

Caller mistakes raise Lua errors: invalid types or options, invalid names or dates, out-of-range indices and inconsistent DataFrame operations.

Filesystem and data failures normally return nil, err:

```
local wb, err = xlsx.open("missing.xlsx")
if not wb then
    io.stderr:write(err, "\n")
end
```

workbook:sheet() follows the same convention for a corrupt or missing sheet part. Workbook:save() checks writing, flushing, closing and publication and no longer reports success after a real write failure.

1.11 Tests and interoperability

./run_tests.sh

The harness covers write/read round trips, Unicode, XML entities, CRC corruption, 1900/1904 dates, styles and number formats, dimensions, frozen panes, filters, formulas, Excel and XML limits, Babet calls, DataFrame copy semantics, collision-free grouping and real openpyxl interoperability.

Babet is resolved in this order: BABET_BIN, the local bin/babet file, then babet from PATH. The harness creates a temporary Python virtual environment, installs openpyxl<=3.1,>4, runs the interoperability round trip separately with Babet and standard Lua when both are available, and removes the venv, pip cache and temporary files at exit, including after a failure. A global openpyxl installation is neither used nor required.

This phase requires python3, the venv module, pip inside the venv and access to the configured Python package index. Debian and Ubuntu systems may need the python3-venv package.

```
cp /path/to/babet bin/babet
chmod +x bin/babet
./run_tests.sh
```

```
BABET_BIN=../babet/bin/babet ./run_tests.sh
LUA_BIN=/usr/bin/lua5.5 ./run_tests.sh
OPENPYXL_SPEC='openpyxl==3.1.5' ./run_tests.sh
```

1.12 Building the PDFs

```
cd doc
./build_doc.sh
```

Outputs:

- documentation-fr.pdf;
- documentation-en.pdf.

Pandoc and XeLaTeX or LuaLaTeX are required. Auxiliary LaTeX files are kept in a temporary directory.

1.13 Known limitations

- Writing uses STORED ZIP entries; reading supports STORED and DEFLATE.
- The internal Lua parser does not support ZIP64.
- No charts, images, data validation, structured tables, or conditional formatting.
- Formulas are read and written but not calculated. Secondary shared formulas without a resolved expression are not written back.
- Style reading covers the supported subset; themes, indexed colors, diagonal borders, and advanced options may be omitted.
- No streaming reader: the file and selected XML parts remain in memory.
- The XML scanner is XLSX-specific, not a general XML parser.
- No Unicode normalization or complete Unicode case folding for sheet names; case-insensitive duplicate detection is reliable for ASCII.